Universidade do Minho
Escola de Engenharia

Relatório 2º Fase | Laboratórios de Informática III

Grupo 52 | 2025/26

Cristiano Gomes (A111836) Duarte Rodrigues (A111517) Mariana Leal (A111248)

Conteúdo

Não foi encontrada nenhuma entrada de índice.

1. Introdução

O presente relatório descreve o trabalho desenvolvido no âmbito da **2.ª Fase do Projeto Final** da unidade curricular **Laboratórios de Informática III**, do 2.º ano da Licenciatura em Engenharia Informática da Universidade do Minho, no ano letivo de **2025/2026**.

Na primeira fase do projeto foi criada uma base de dados em memória a partir dos ficheiros CSV fornecidos, contendo informação sobre aeroportos, voos, aeronaves, passageiros e reservas. Esta fase centrou-se na leitura, validação e armazenamento dos dados, na implementação do modo batch através do programa principal e na resolução das três queries obrigatórias definidas no enunciado.

A segunda fase teve como objetivo a expansão e consolidação da aplicação, abrangendo a implementação da totalidade das seis queries, a adaptação ao novo dataset com maior volume de dados e o desenvolvimento de um **modo interativo**, permitindo a execução dinâmica das queries pelo utilizador.

Ao longo do projeto foram aplicados conceitos fundamentais da engenharia de software, como **modularidade, encapsulamento, abstração e reutilização de código**, recorrendo a estruturas de dados adequadas e eficientes. Foram ainda utilizadas bibliotecas e ferramentas relevantes para o desenvolvimento em C, nomeadamente a **glib 2.0**, bem como ferramentas de depuração e análise de memória, como o **Valgrind**.

Com esta nova fase surgiram desafios adicionais relacionados com o **desempenho do programa**, tornando-se essencial garantir não só a correção funcional, mas também a eficiência em termos de tempo de execução e consumo de memória.

Este relatório apresenta, assim, as principais decisões e estratégias adotadas na segunda fase do projeto, bem como uma análise crítica do trabalho desenvolvido e dos resultados obtidos.

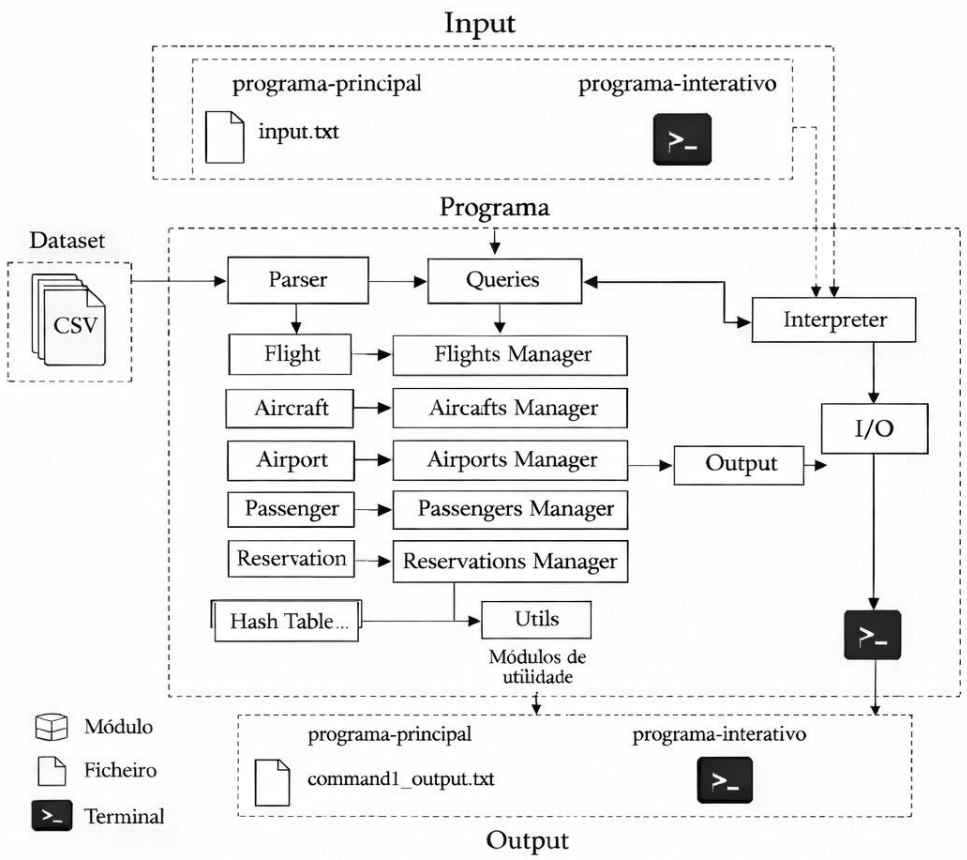
2. Sistema

A arquitetura do sistema desenvolvido nesta fase foi organizada de forma modular, separando claramente os componentes responsáveis pela leitura, validação, armazenamento e consulta dos dados. O programa foi estruturado em diversos módulos independentes, permitindo uma gestão eficiente das entidades (aeroportos, voos, aeronaves, passageiros e reservas) e garantindo a flexibilidade necessária para evoluções futuras do projeto.

No projeto deste ano foram implementados o parsing e a validação dos dados de entrada, bem como o modo batch através do programa-principal, responsável por interpretar o ficheiro de comandos e executar sequencialmente as queries definidas. Este processo envolve não só a leitura e interpretação dos diferentes ficheiros CSV, mas também o armazenamento organizado dos dados em memória, recorrendo a estruturas adequadas para garantir eficiência no acesso a cada entidade.

Para suportar esta organização, foi utilizada a biblioteca glib, que disponibiliza estruturas de dados robustas — como tabelas de hash e listas ligadas — facilitando a gestão, consulta e indexação da informação carregada.

No desenvolvimento do trabalho, utilizou-se a seguinte estrutura:



Arquitetura do Projeto

2.1. Managers

Os *managers* são os módulos responsáveis por armazenar, gerir e disponibilizar os dados carregados a partir dos ficheiros CSV. Cada entidade do sistema — **voos, aeroportos, aeronaves, passageiros e reservas** — possui um *manager* dedicado, que encapsula as respetivas estruturas de dados e fornece uma interface controlada para acesso e manipulação da informação.

Na segunda fase do projeto, estes módulos foram estendidos para suportar um maior volume de dados e responder a requisitos adicionais das novas queries. Para tal, recorreu-se de forma sistemática a **tabelas de hash da biblioteca glib**, garantindo operações eficientes de inserção, pesquisa e iteração. Em alguns casos, foram também introduzidas estruturas auxiliares (como arrays ou contadores agregados) com o objetivo de otimizar o desempenho de consultas frequentes.

O encapsulamento dos dados nos *managers* revelou-se fundamental nesta fase, permitindo a evolução interna das estruturas sem impacto direto nos restantes módulos do sistema, nomeadamente nas queries e no modo interativo.

2.2. Parsing

Os *parsers* do projeto encontram-se organizados por entidade, existindo um módulo específico para cada ficheiro CSV do dataset: flights_parser, airports_parser, aircrafts_parser, passengers_parser e reservations_parser. Cada um destes módulos utiliza um parser genérico para efetuar a leitura sequencial do ficheiro, dividir os campos e preencher estruturas temporárias adequadas ao tipo de dados em processamento.

Durante esta fase são aplicadas todas as **validações sintáticas** definidas no enunciado, assegurando a correção de formatos de datas, códigos IATA, campos numéricos, listas, URLs e restantes atributos. Sempre que necessário, os valores são normalizados, removendo aspas, convertendo tipos e preparando a informação para validações posteriores.

Na segunda fase, o processo de parsing foi reforçado para integrar **validações lógicas mais complexas**, dependentes de relações entre entidades, como a existência de voos associados a reservas, a coerência temporal entre horários estimados e reais, ou o encadeamento correto de voos numa reserva com múltiplos segmentos. Sempre que ocorre um erro sintático ou lógico, a linha original é registada no respetivo ficheiro _errors.csv, garantindo um controlo rigoroso das entradas inválidas.

2.3. Queries

Os módulos responsáveis pelas queries foram alargados na segunda fase do projeto, passando a abranger a totalidade das seis interrogações definidas no enunciado. Cada query recebe os seus argumentos já processados pelo parser de comandos e acede às estruturas de dados disponibilizadas pelos managers das diferentes entidades.

Os módulos responsáveis pelas queries foram estendidos na segunda fase do projeto, passando a suportar as queries 5 e 6, que introduzem operações de agregação e análise mais complexas. Cada query recebe os

seus argumentos já validados pelo parser de comandos e acede às estruturas de dados disponibilizadas pelos *managers* das diferentes entidades.

A Query 5 envolve o cálculo do atraso médio por companhia aérea, recorrendo a contadores e acumuladores para processar apenas voos com estado *Delayed*, permitindo um cálculo eficiente das médias e a posterior ordenação dos resultados. A Query 6 determina o aeroporto de destino mais frequente para passageiros de uma determinada nacionalidade, cruzando informação entre passageiros, reservas e voos, através de mecanismos de contagem agregada por aeroporto.

Em ambas as queries, foi dada especial atenção à eficiência dos percursos sobre as estruturas em memória e à reutilização de dados. Os módulos de queries foram ainda concebidos de forma independente do modo de execução, permitindo a sua utilização tanto no modo batch como no modo interativo.

Esta organização contribui para um código mais modular, eficiente e alinhado com os princípios de modularidade e encapsulamento exigidos na segunda fase do projeto.

3. Discussão

3.1. Modularização e Encapsulamento

Na **primeira fase do projeto**, foi definida a arquitetura base do sistema, assente na separação entre *managers* e entidades, o que permitiu uma implementação mais organizada e de fácil manutenção. Cada *manager* ficou responsável pela lógica de armazenamento e acesso aos dados — voos, aeroportos, aeronaves, passageiros e reservas — enquanto as entidades se limitaram a representar as estruturas de dados, promovendo um encapsulamento claro e reduzindo dependências diretas entre módulos.

Na **segunda fase**, esta arquitetura foi consolidada e estendida para acomodar o aumento do número de queries e a introdução do **modo interativo**. A manutenção da separação entre componentes permitiu reutilizar as mesmas estruturas e interfaces nos diferentes modos de execução, absorvendo a complexidade adicional sobretudo ao nível dos *managers* e dos módulos de queries, sem impactar as entidades.

Com o crescimento do sistema, a modularização exigiu um maior cuidado na definição e estabilidade das interfaces expostas pelos módulos. Apesar deste desafio, esta abordagem contribuiu para um código mais robusto, extensível e coerente com os princípios de **modularidade e encapsulamento** exigidos na avaliação da segunda fase do projeto.

3.2. Estruturas de Dados

Na segunda fase do projeto, as principais estruturas de dados utilizadas foram a **GHashTable** e o **GArray**, fornecidas pela biblioteca **GLib**, adequadas ao aumento do volume de dados e à complexidade das queries implementadas. A **GHashTable** foi usada para armazenar entidades indexadas por identificadores únicos, como aeroportos, voos, aeronaves, passageiros e reservas, permitindo operações de inserção e consulta eficientes, com complexidade média próxima de **O(1)**.

O **GArray** foi utilizado como estrutura complementar para armazenar coleções de elementos de forma contígua, facilitando iterações e agregações intermédias necessárias ao cálculo de métricas globais. A utilização da **GLib** contribuiu ainda para uma gestão mais segura da memória, reduzindo o risco de *memory leaks*.

Estas escolhas revelaram-se adequadas aos requisitos da segunda fase, oferecendo um compromisso equilibrado entre **simplicidade, desempenho e consumo de memória**.

3.3. Execução das Queries

3.3.1. Query 1

A **Query 1** recebe o código IATA de um aeroporto e verifica a sua existência no AirportsManager. A pesquisa é feita diretamente na tabela de hash onde os aeroportos válidos foram previamente inseridos durante o parsing dos CSV. Se o aeroporto existir, o sistema recolhe os seus campos principais — código, nome, cidade, país e tipo — e constrói a linha final no formato exigido pelo enunciado. Caso contrário, devolve simplesmente uma linha vazia, uma vez que o código não corresponde a nenhum aeroporto válido carregado.

3.3.2. Query 2

A **Query 2** percorre todas as aeronaves presentes no AircraftsManager e calcula, para cada uma delas, o número total de voos não cancelados registados no FlightsManager. Para isso, o sistema itera sobre todos os voos armazenados e compara o identificador da aeronave de cada voo com o identificador atual. Caso seja fornecido um fabricante como filtro opcional, o sistema ignora todas as aeronaves cujo fabricante não corresponda ao argumento recebido. Depois de obtidos os contadores, é efetuada uma ordenação: primeiro por número de voos (de forma decrescente) e, em caso de empate, por ordem lexicográfica crescente do identificador. Os primeiros N elementos da lista ordenada são então convertidos no formato final contendo identificador, fabricante, modelo e número de voos realizados.

3.3.3. Query 3

A **Query 3** determina qual o aeroporto com mais partidas dentro de um intervalo de datas, analisando exclusivamente os voos cuja data real de partida (actual departure) esteja compreendida no período definido. Para comparar datas, é utilizada uma comparação direta com strncmp, uma vez que o formato YYYY-MM-DD permite ordenação lexicográfica. Apenas voos com estado diferente de Cancelled são considerados. Durante o processamento, é mantido um contador para cada aeroporto de origem, acumulando o número de partidas válidas. Depois de percorrer todos os voos, o sistema escolhe o aeroporto com maior número de partidas e, em caso de empate, seleciona o código IATA menor por ordem lexicográfica. A linha final inclui o código, nome, cidade, país e número total de partidas. Se não existir nenhum voo válido dentro do intervalo, é devolvida uma linha vazia.

3.3.4. Query 4

A **Query 4** tem como objetivo identificar o passageiro que permaneceu mais tempo no *top 10* dos passageiros com maior valor gasto em viagens ao longo do tempo. O cálculo baseia-se na agregação semanal do valor total gasto por cada passageiro, considerando a soma dos preços das reservas associadas aos seus voos. As semanas são delimitadas com início ao domingo e término ao sábado, sendo utilizada a data estimada de partida (*departure*) para a determinação do período temporal.

Conceptualmente, esta query requer o cruzamento de informação entre passageiros, reservas e voos, bem como a manutenção de estruturas intermédias para agregação de valores por passageiro e por semana. Para cada semana considerada, é determinado o *top 10* de passageiros com maior despesa acumulada, incrementando um contador associado a cada passageiro sempre que este integra esse conjunto. No final do processamento, é selecionado o passageiro com maior número de ocorrências no *top 10*, sendo utilizado o número de documento como critério de desempate.

3.3.5. Query 5

A **Query 5** calcula o *top N* de companhias aéreas com maior tempo médio de atraso por voo. Para esse efeito, o sistema percorre todos os voos armazenados no *FlightsManager*, considerando exclusivamente aqueles cujo estado é **Delayed**. Para cada voo válido, é calculado o atraso em minutos, correspondente à diferença entre a data real de partida (*actual departure*) e a data estimada (*departure*).

Durante o processamento são mantidos contadores e acumuladores por companhia aérea, permitindo calcular o número total de voos em atraso e o somatório do tempo de atraso. Após o percurso completo dos voos, é calculado o atraso médio por companhia aérea. Os resultados são ordenados por atraso médio de forma decrescente e, em caso de empate, por ordem alfabética do nome da companhia, sendo apresentados os primeiros *N* elementos no formato definido.

3.3.6. Query 6

A **Query 6** determina o aeroporto de destino mais frequente para passageiros de uma determinada nacionalidade. O sistema começa por identificar os passageiros cuja nacionalidade corresponde ao argumento fornecido e percorre as reservas associadas, analisando os voos correspondentes. Apenas são considerados voos com estado diferente de **Cancelled**.

Durante o processamento é mantido um contador agregado por aeroporto de destino, permitindo contabilizar o número de passageiros que aterraram em cada aeroporto. Após a análise de todos os dados relevantes, é selecionado o aeroporto com o maior número de chegadas. Em caso de empate, é escolhido o aeroporto com o menor código IATA em ordem lexicográfica. Caso não existam passageiros com a nacionalidade indicada, é devolvida uma linha vazia.

3.4. Programa Interativo

O programa interativo, desenvolvido na segunda fase do projeto, permite ao utilizador interagir de forma dinâmica com o sistema, escolhendo a query a executar e introduzindo os respetivos parâmetros de forma guiada. Este modo foi concebido com o objetivo de tornar a utilização do programa mais intuitiva, fornecendo indicações claras sobre os dados necessários para a correta execução de cada query.

Durante o carregamento do dataset, são aplicados os mecanismos de validação implementados no projeto, garantindo que apenas dados consistentes e de acordo com as restrições definidas são armazenados em memória. Estas validações abrangem diferentes tipos de informação, como identificadores, datas e instantes temporais, códigos normalizados e valores numéricos com limites específicos, permitindo detetar e rejeitar registos inválidos logo na fase de parsing.

Adicionalmente, no modo interativo, sempre que o utilizador introduz um input inválido para uma determinada query, o sistema valida os dados fornecidos e apresenta uma mensagem de erro adequada. Esta abordagem contribui para uma interação mais fiável e robusta, prevenindo execuções incorretas e assegurando a consistência dos resultados obtidos.

4. Conclusão

Ao longo da **primeira fase do projeto**, foi definida e implementada a arquitetura base do sistema, centrada na separação entre entidades e *managers*, bem como na leitura, validação e armazenamento dos dados provenientes dos ficheiros CSV. Esta fase permitiu consolidar os fundamentos do projeto e garantir uma base sólida e funcional para a evolução do sistema.

Na **segunda fase**, o projeto foi significativamente expandido, introduzindo novas queries, o modo interativo e uma maior preocupação com o **encapsulamento, desempenho e gestão de memória**. Surgiram desafios relevantes, nomeadamente ao nível da organização entre módulos e da necessidade de manter o tempo de execução e o consumo de memória dentro dos limites estabelecidos, especialmente face ao aumento do volume de dados. Estas dificuldades foram ultrapassadas através da reorganização do código, da definição mais rigorosa das interfaces e da utilização criteriosa de estruturas auxiliares, libertadas após a execução de cada query.

De um modo geral, o desenvolvimento do projeto permitiu uma evolução significativa no domínio da **linguagem C** e das ferramentas associadas, bem como uma maior consciência da importância da **modularidade, encapsulamento e eficiência computacional** no desenvolvimento de aplicações de maior dimensão. A experiência adquirida ao longo das duas fases contribuiu para uma visão mais completa do processo de desenvolvimento de software, aliando correção funcional a desempenho e robustez.